



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica Scienza e Ingegneria

Corso di Laurea in Informatica

Traduzione da linguaggio naturale a query per Blazegraph

Relatore:
Chiar.mo Prof.
Fabio Vitali

Presentata da:
Alberto Zuccari

I Sessione 2026
Anno Accademico 2025/2026

(DA FARE ALLA FINE)

5 parole chiave per caratterizzare il contenuto della dissertazione:

parola 5

parola 4

parola 3

parola 2

Parola 1

*Non tocca a noi dominare tutte le maree del mondo
il nostro compito è di fare il possibile
per la salvezza degli anni nei quali viviamo*

- J.R.R. Tolkien

Abstract

Abstract qui (da completare alla fine).

Indice

1	Introduzione	1
1.1	Il contesto e il problema	1
1.2	Struttura della dissertazione	1
2	Contesto scientifico e tecnologico	3
2.1	Introduzione al capitolo	3
2.2	Stato dell'arte: dall'interrogazione formale alla ricerca semantica . . .	3
2.2.1	Il contesto: Digital Humanities e la barriera della ricerca . . .	3
2.2.2	Il divario tra ricerca strutturata e ricerca esplorativa	4
2.2.3	La "scatola vuota" e le tre barriere di SPARQL	4
2.3	Fondamenti di architettura dell'informazione	5
2.3.1	L'illusione della ricerca algoritmica	5
2.3.2	Recall, Precision e il compromesso ineluttabile	5
2.3.3	Carico cognitivo: la Legge del minimo sforzo e la Legge di Hick .	6
2.3.4	Paradigma classico: navigazione e filtri rigidi	7
2.3.5	Il nuovo paradigma: ricerca semantica con LLM	7
2.4	Retrieval-Augmented Generation (RAG) come soluzione tecnologica .	8
2.4.1	Il paradigma RAG tradizionale	8
2.4.2	RAG applicato a Knowledge Graphs	8
2.4.3	Fine-tuning e LoRA per Domain Adaptation	9
2.5	Conclusioni del capitolo	9
3	Il prototipo	11
3.1	Introduzione al capitolo	11
3.2	Il dominio e gli utenti target	11
3.2.1	Il contesto operativo: DH.ARC e il Datavault	11
3.2.2	Profilo dell'utente finale: i ricercatori del centro	12
3.3	Architettura Funzionale (Ad alto livello)	12
3.3.1	Il flusso dell'interazione	12
3.3.2	Diagramma funzionale del sistema	13
3.3.3	I componenti del sistema e le loro responsabilità	13
3.4	Casi d'uso principali	13
3.4.1	Caso d'uso 1: ricerca per caratteristiche di acquisizione	13
3.4.2	Caso d'uso 2: ricerca per progetto e tipologia di risorsa	14
3.4.3	Caso d'uso 3: ricerca multi-criterio con filtro temporale	14

3.5	Conclusioni del capitolo	14
4	Architettura e dettagli implementativi	17
4.1	Introduzione al capitolo	17
4.2	Stack tecnologico: Fedora e Blazegraph	17
4.3	Pipeline di fine-tuning dell'LLM	17
4.3.1	Dataset generativo e schema SPARQL	18
4.3.2	Configurazione dell'addestramento con Unsloth	18
4.4	Architettura del backend e integrazione frontend	18
4.4.1	Inference engine (FastAPI)	18
4.4.2	Integrazione con Blazegraph tramite proxy Vite	18
4.5	Formato dati: dataset_datavault_v4.jsonl	19
4.5.1	Le 8 regole di generazione SPARQL	19
4.5.2	Gestione degli errori: OUT_OF_DOMAIN	19
4.6	Conclusioni del capitolo	19
5	Valutazione	21
5.1	Introduzione all'approccio valutativo	21
5.2	Valutazione dell'efficienza (Quantitativa)	21
5.2.1	Ambienti di test: GPU vs CPU	21
5.2.2	Impronta in memoria e throughput	21
5.3	Valutazione dell'efficacia (Qualitativa)	21
5.3.1	Precision e Recall su Blazegraph	21
5.3.2	Analisi del tasso di allucinazione	21
5.3.3	Robustezza sui casi limite (Edge Cases)	21
5.4	Conclusioni del capitolo	21
6	Conclusioni e sviluppi futuri	23
6.1	Risultati raggiunti	23
6.2	Motivazioni di orgoglio e modestia	23
6.2.1	Punti di forza e orgoglio	23
6.2.2	Limiti del sistema e punti di modestia	23
6.3	Sviluppi futuri	23
6.3.1	Bug fix e ottimizzazioni a breve termine	23
6.3.2	Estensioni fondamentali e feature mancanti	23
6.3.3	Visioni di lungo termine (Desideri e sogni)	23
	Riferimenti bibliografici	25

Elenco delle tabelle

Elenco delle figure

Capitolo 1

Introduzione

1.1 Il contesto e il problema

1.2 Struttura della dissertazione

Capitolo 2: Contesto scientifico e tecnologico

Capitolo 3: Il prototipo

Capitolo 4: Architettura e implementazione

Capitolo 5: Valutazione

Capitolo 6: Conclusioni e sviluppi futuri

Capitolo 2

Contesto scientifico e tecnologico

2.1 Introduzione al capitolo

Il capitolo esplora il background scientifico e tecnologico alla base del lavoro proposto. Partiamo dal *problema fondamentale*: come fornire accesso content-aware a repository RDF di dimensioni massive per utenti privi di competenze tecniche formali?

La risposta richiede una comprensione su tre livelli di complessità:

1. **Lo stato dell'arte**: come gli approcci precedenti (form rigidi, endpoint SPARQL nudi) falliscono nel dominio umanistico;
2. **I fondamenti teorici**: i principi dell'architettura dell'informazione che guidano un'alternativa;
3. **Le tecnologie emergenti**: come Retrieval-Augmented Generation (RAG) e Large Language Models (LLM) abilitano una soluzione più flessibile.

Procederemo così: nel paragrafo 2.2 analizziamo il divario tra i bisogni esplorativi degli studiosi umanisti e la rigidità sintattica dei sistemi tradizionali. Successivamente, nel paragrafo 2.3, esaminiamo i principi fondativi dell'architettura dell'informazione che giustificano il ricorso agli LLM. Infine, nel paragrafo 2.4, descriviamo il paradigma RAG come soluzione tecnologica che bilancia flessibilità e affidabilità.

2.2 Stato dell'arte: dall'interrogazione formale alla ricerca semantica

2.2.1 Il contesto: Digital Humanities e la barriera della ricerca

Nel dominio delle Digital Humanities (DH) e della gestione del patrimonio culturale, la ricerca all'interno di repository RDF rappresenta un ostacolo significativo per chi manca di competenze tecniche avanzate. Come evidenziato in letteratura, lavorare con strumenti come SPARQL impone vincoli rigorosi[VFQP25] e richiede una familiarità profonda con ontologie complesse e convenzioni di metadati[Vaš24].

Tali approcci si basano su query create manualmente, che risultano fragili e difficili da adattare a esigenze mutevoli[Mar25]. Questo pone il ricercatore di fronte all’ostacolo cognitivo della “scatola vuota”: chi possiede un bisogno informativo vago o in via di definizione viene paralizzato dalla necessità di dichiarare tutto a priori.

2.2.2 Il divario tra ricerca strutturata e ricerca esplorativa

Alla base dei tradizionali sistemi di Information Retrieval (IR) vi è un assunto fondamentale: che l’utente inizi l’interazione con un bisogno informativo perfettamente strutturato. Nel dominio umanistico, tuttavia, l’utente procede per esplorazione.

Questo comportamento è stato teorizzato da Marcia Bates con il modello *berry-picking*[Bat89]: una strategia di ricerca non lineare in cui gli studiosi si muovono iterativamente, aggiustando la rotta e raffinando gli obiettivi man mano che acquisiscono nuove informazioni durante l’esplorazione. In un tale scenario di ricerca esplorativa, assume un valore inestimabile il fenomeno della *serendipity*, la scoperta inattesa di pattern rilevanti durante la navigazione dei dati, che gli strumenti tradizionali non incoraggiano, anzi, spesso ostacolano.

Considerando il netto contrasto tra la rigidità sintattica di SPARQL e la natura fluida ed evolutiva della ricerca umanistica, emerge con chiarezza la necessità di un nuovo paradigma di interazione, che consenta ai ricercatori di esprimere bisogni emergenti senza dover ricorrere a linguaggi formali.

2.2.3 La “scatola vuota” e le tre barriere di SPARQL

Costringere ricercatori umanisti a esprimere bisogni complessi tramite endpoint SPARQL genera un ostacolo su tre livelli:

1. **Barriera cognitiva:** L’interazione esige la scrittura in un linguaggio dichiarativo formale e l’obbligo di conoscere complessi prefissi ontologici (es. `dc:`, `exif:`, `ebucore:`) solo per abbozzare una query di base.
2. **Barriera strutturale:** L’assenza di una mappa visiva delle relazioni all’interno del grafo RDF fa sì che l’utente non sappia quali proprietà esistano realmente nel database né come i dati siano formalizzati nell’ontologia.
3. **Barriera pratica:** La forte rigidità sintattica determina che un singolo errore di battitura produca un set di risultati vuoto. L’assenza di messaggi di errore parlanti impedisce qualsiasi tentativo di debugging autonomo.

Studi sull’usabilità delle interfacce per Knowledge Graphs confermano che linguaggi come SPARQL impongono vincoli rigidi e richiedono una conoscenza esplicita degli schemi, competenze estranee ai professionisti del patrimonio culturale[VFQP25].

2.3 Fondamenti di architettura dell'informazione

La progettazione di un sistema di accesso per un database RDF non può limitarsi alla pura efficienza computazionale del recupero dati. Deve invece rispondere ai principi fondamentali dell'*architettura dell'informazione*, ovvero la disciplina che studia come rendere le informazioni realmente trovabili e usabili.

In un contesto umanistico, costringere l'utente a interfacciarsi con la complessità dello schema sottostante attraverso filtri rigidi o linguaggi formali genera un ostacolo cognitivo. Procediamo ora ad analizzare i principi teorici che giustificano il ricorso a un'interfaccia conversazionale mediata da LLM.

2.3.1 L'illusione della ricerca algoritmica

I sistemi di reperimento basati su query formali (SQL, SPARQL) assumono erroneamente che l'utente sappia sempre cosa cercare e come tradurre il proprio bisogno in un costrutto logico. Peter Morville e Louis Rosenfeld hanno smontato questo assunto come un grave errore concettuale:

“Consideriamo questo modello pericoloso perché basato su un equivoco: che trovare informazioni sia un problema diretto, che può essere risolto da un semplice approccio algoritmico. Dopo tutto, abbiamo risolto la sfida del recupero dati — che, ovviamente sono fatti e cifre — con tecnologie database come SQL. Così procede la teoria, si trattano idee e concetti incorporati nei documenti testuali semistrutturati nello stesso modo”
[MR08]

I sistemi algoritmici tradizionali falliscono nel dominio umanistico perché sono progettati per recuperare *dati esatti*, mentre gli studiosi ricercano *concetti, idee e relazioni*. Tali concetti sono per loro natura ambigui e sfumati. Un intermediario LLM colma esattamente questo divario: funge da traduttore semantico che accetta l'ambiguità e la ricchezza del linguaggio naturale e la converte in modo trasparente nell'approccio algoritmico rigido richiesto dalla macchina.

2.3.2 Recall, Precision e il compromesso ineluttabile

Un problema critico delle interfacce tradizionali a filtri è l'impossibilità di calibrare dinamicamente il raggio della ricerca. Morville e Rosenfeld spiegano che ogni ricerca è un compromesso matematico tra due metriche inversamente correlate:

$$\text{recall} = \frac{|\text{doc rilevanti recuperati}|}{|\text{doc nella raccolta}|} \quad (2.1)$$

$$\text{precision} = \frac{|\text{doc rilevanti recuperati}|}{|\text{doc rilevanti della raccolta}|} \quad (2.2)$$

“[...] Non sarebbe bello poter ottenere contemporaneamente elevati valori di recall e precision? Purtroppo non si può avere botte piena e moglie ubriaca: recall e precision sono inversamente correlati”
[MR08]

In un'interfaccia a maschera con decine di campi e filtri ontologici, l'utente è costretto a dichiarare a priori il livello di precisione desiderato. Se sbaglia un filtro, ottiene zero risultati (eccesso di Precision); se ne omette uno, viene sommerso da migliaia di risultati irrilevanti (eccesso di Recall).

In una conversazione in linguaggio naturale con un LLM, invece, l'aggiustamento avviene in modo iterativo e fluido. L'utente può iniziare con una richiesta esplorativa e ampia (es. “Mostrami tutti i manoscritti del XIV secolo”, privilegiando il Recall) e, osservando i primi risultati, raffinare il tiro conversando (“Filtra solo quelli con miniature e digitalizzati ad alta risoluzione”, aumentando la Precision). L'LLM permette di variare queste metriche dinamicamente senza costringere l'utente a decostruire e riscrivere manualmente una complessa sintassi di query.

2.3.3 Carico cognitivo: la Legge del minimo sforzo e la Legge di Hick

Dal punto di vista del carico cognitivo, esporre all'utente l'immensa topologia di un Knowledge Graph attraverso decine di menu a tendina o l'obbligo di conoscere specifiche ontologie porta alla frustrazione potrebbe sfociare nell'abbandono del sistema.

Luca Rosati analizza questo fenomeno attraverso due principi cognitivi fondamentali. Il primo è la *legge del minimo sforzo*:

“La legge del minimo sforzo (least effort) regola quindi il nostro sistema di information seeking, al punto che spesso accettiamo anche contenuti di bassa qualità o affidabilità (se più facili da ottenere e usare) pur di non ricorrere a strategie di ricerca attive, le quali comportano necessariamente sforzi, tempi e competenze maggiori” [Ros07]

Se l'interfaccia richiede la fatica di apprendere prefissi RDF o di comprendere uno schema dati complesso, è plausibile che l'utente rinuncerà alla ricerca.

Il secondo principio è la *Legge di Hick*, la quale afferma che, in presenza di n alternative equiprobabili, il tempo necessario per selezionarne una è proporzionale (a meno di una costante additiva) al logaritmo in base 2 del numero di scelte più 1:

$$\text{tempo} = a + b \log_2(n + 1)$$

I coefficienti a e b dipendono da specifiche condizioni al contorno, tra cui le modalità di presentazione delle opzioni e il grado di familiarità dell'utente [Ros07].

Tuttavia, Rosati sottolinea che, qualora le alternative “siano presentate in modo confuso” o costituiscano “una lista disordinata” priva di un criterio logico riconoscibile, il tempo di decisione scala in modo lineare, aumentando il carico cognitivo fino a divenire insostenibile e causando paralisi decisionale. Esporre l'utente a un form SPARQL nudo o a una maschera di ricerca avanzata con decine di menu a tendina equivale esattamente a questo: innescare un “paradosso della scelta” paralizzante.

Un'interfaccia conversazionale basata su LLM nasconde questa complessità strutturale nel backend. L'utente si trova di fronte a un singolo "spazio" (un box di chat), azzerando l'effetto della Legge di Hick e assecondando la Legge del minimo sforzo. Non dovendo scegliere tra decine di opzioni incomprensibili, il ricercatore delega il carico cognitivo della traduzione ontologica all'LLM, dedicando le proprie energie intellettuali esclusivamente all'analisi e alla scoperta delle risorse.

2.3.4 Paradigma classico: navigazione e filtri rigidi

L'architettura dell'informazione si definisce come il design strutturale di ambienti informativi finalizzato a plasmare l'usabilità e la trovabilità[MR08]. Poggia su quattro sistemi interconnessi:

- (i) sistemi di organizzazione, che categorizzano le informazioni;
- (ii) sistemi di etichettatura, che ne determinano la rappresentazione linguistica;
- (iii) sistemi di navigazione, che consentono l'orientamento nello spazio;
- (iv) sistemi di ricerca, che permettono interrogazioni dirette.

Applicato al Datavault, l'interrogazione tradizionale mediante filtri rigidi o SPARQL presenta tre limiti strutturali:

1. Pretende conoscenza a priori di stringhe esatte, incompatibile con la ricerca umanistica [Vaš24];
2. Non gestisce sinonimia, sfumature e vaghezza del linguaggio naturale;
3. Obbliga la formulazione di query multi-criterio in sintassi rigide.

Queste rigidità abbattano la "trovabilità": l'informazione esiste fisicamente ma rimane irraggiungibile per l'utente.

2.3.5 Il nuovo paradigma: ricerca semantica con LLM

L'introduzione di un LLM modifica radicalmente il paradigma di esplorazione dei dati semantici, agendo come un intermediario e traduttore capace di convertire fluidamente il linguaggio naturale in interrogazioni strutturate come query SPARQL.

Questa trasformazione definisce un concetto chiave nell'interazione uomo-dato: non è più il ricercatore a dover imparare il linguaggio della macchina, ma è il sistema stesso che si adatta all'utente, comprendendone le richieste nel formato linguistico naturale. Ne consegue che questo approccio sposta l'intero carico di complessità cognitiva e sintattica dall'utente al sistema computazionale.

Questo trasferimento della complessità si lega esplicitamente al concetto di "findability" (trovabilità)[Ros07]. Lo scopo del nuovo sistema non è semplicemente rendere una risorsa formalmente accessibile, ma garantire che essa risulti effettivamente trovabile dal ricercatore nel momento del bisogno.

La ricerca semantica mediata dall'LLM aumenta in modo decisivo la trovabilità per tre ragioni:

- abbassa drasticamente la soglia di accesso all'informazione, permettendo a chiunque di fare una domanda diretta al database;
- il modello gestisce in modo nativo la vaghezza, l'ambiguità e i sinonimi intrinseci nel linguaggio umano;
- consente di esprimere bisogni informativi multi-criterio complessi, impossibili da supportare attraverso form a tendina.

Mantenendo un doveroso grado di obiettività scientifica, occorre tuttavia dichiarare che il paradigma basato su LLM presenta sfide intrinseche: il rischio di allucinazioni (invenzione di predicati inesistenti), la dipendenza dall'accuratezza del prompt di sistema, e l'elevato costo computazionale delle chiamate ai modelli generativi. Queste metriche verranno analizzate in dettaglio nel capitolo 5.

2.4 Retrieval-Augmented Generation (RAG) come soluzione tecnologica

2.4.1 Il paradigma RAG tradizionale

Retrieval-Augmented Generation (RAG) è un paradigma che combina la potenza dei modelli generativi con l'affidabilità del recupero di informazioni da fonti strutturate. La struttura di base è semplice ma efficace:

1. **Ingresso:** l'utente pone una domanda in linguaggio naturale;
2. **Retrieval:** un modulo di ricerca estrae documenti o frammenti rilevanti da una base di conoscenza strutturata;
3. **Augmentation:** il contesto recuperato viene fornito al modello generativo come "prompt esteso";
4. **Generation:** il modello generativo produce una risposta grounded nei documenti recuperati, riducendo le allucinazioni.

Questo paradigma riduce significativamente il rischio di allucinazioni perché il modello è vincolato a operare all'interno di uno spazio di conoscenza ben definito.

2.4.2 RAG applicato a Knowledge Graphs

Nel nostro caso, il "corpus" da cui recuperare informazioni è uno **schema RDF strutturato**. L'applicazione di RAG a Knowledge Graphs richiede una variante: anziché recuperare documenti di testo, recuperiamo *frammenti di schema, esempi di predicati, e descrizioni di ontologie*.

Quindi il flusso diventa:

1. **Ingresso:** domanda dell'utente (es. "Mostrami tutte le immagini digitalizzate con una Nikon nel 2022");
2. **Retrieval:** il sistema accede allo schema RDF del Datavault e recupera i predicati rilevanti (es. `exif:make`, `exif:dateTime`);
3. **Augmentation:** lo schema e gli esempi di predicati vengono forniti al modello LLM come parte del prompt di sistema;
4. **Generation:** il modello LLM produce una query SPARQL coerente con lo schema;
5. **Esecuzione:** la query viene eseguita su Blazegraph e i risultati vengono restituiti all'utente.

Questo approccio combina i lati positivi di entrambi: la flessibilità del linguaggio naturale e l'affidabilità di una fonte di conoscenza strutturata e verificabile.

2.4.3 Fine-tuning e LoRA per Domain Adaptation

Un miglioramento ulteriore consiste nel fine-tuning del modello LLM su un dataset specifico del dominio. Anziché usare un LLM generico, possiamo adattare un modello più piccolo (es. `Phi-3-mini`) a generare query SPARQL corrette per il Datavault specifico.

Tecniche come LoRA (*Low-Rank Adaptation*) permettono un fine-tuning efficiente senza memorizzare l'intero modello in memoria. Il modello apprende:

1. La sintassi SPARQL corretta per il Datavault;
2. L'ontologia specifica (predicati, tipi di dato, convenzioni di naming);
3. Strategie di generazione robuste (evitare predicati inesistenti, gestire date, etc.);
4. Quando e come segnalare query out-of-domain.

Questo approccio riduce significativamente il rischio di allucinazioni e aumenta l'accuratezza della generazione di query.

2.5 Conclusioni del capitolo

In questo capitolo abbiamo analizzato il contesto scientifico e tecnologico che motiva il nostro contributo. Abbiamo visto che:

1. Gli approcci tradizionali (SPARQL nudo, filtri rigidi) falliscono nel dominio umanistico perché impongono barriere cognitive, strutturali e pratiche;
2. I modelli teorici dell'architettura dell'informazione (berrypicking, precision e recall, carico cognitivo) giustificano il ricorso a un'interfaccia conversazionale mediata da LLM;

3. Le tecnologie emergenti (RAG, fine-tuning con LoRA) rendono possibile un'implementazione affidabile e accurata di tale interfaccia.

Avendo chiarito il fondamento scientifico e tecnologico della nostra soluzione, il capitolo 3 presenta nel concreto il nostro LLM Query Builder: il dominio di applicazione (DH.ARC e il Datavault), il profilo degli utenti target, l'architettura funzionale del sistema, e i casi d'uso che dimostrano il valore aggiunto della nostra proposta nel contesto umanistico.

Capitolo 3

Il prototipo

3.1 Introduzione al capitolo

Avendo chiarito il fondamento scientifico e tecnologico nel capitolo 2, presentiamo ora il prototipo di LLM Query Builder realizzato come soluzione concreta ai problemi identificati.

Questo capitolo è organizzato secondo una prospettiva *user-centered*: partiamo dal dominio di applicazione (DH.ARC e il Datavault) e dal profilo degli utenti target (paragrafo 3.2), passiamo all’architettura funzionale del sistema come appare all’utente (paragrafo 3.3), e concludiamo con tre scenari d’uso concreti (paragrafo 3.4) che dimostrano il valore aggiunto della nostra proposta.

3.2 Il dominio e gli utenti target

3.2.1 Il contesto operativo: DH.ARC e il Datavault

Il Digital Humanities Advanced Research Centre (DH.ARC) è un polo di ricerca dell’Università di Bologna afferente al Dipartimento di Filologia Classica e Italianistica (FICLIT), nato con l’obiettivo di “connettere studenti, ricercatori, personale IT e professori del FICLIT e del Dipartimento di Informatica — Scienza e Ingegneria (DISI)”[ALM26].

Il centro ha lo scopo di fungere da hub per gli studiosi e le istituzioni, supportandoli “nella progettazione, sviluppo e manutenzione di progetti di ricerca DH” e promuovendo iniziative innovative nel campo umanistico.

Le attività di ricerca vertono prevalentemente sulla digitalizzazione e creazione di metadati per il patrimonio culturale, che includono in gran parte “documenti archivistici e libri antichi”, “testi letterari” e “knowledge graphs che descrivono oggetti del patrimonio culturale” come fotografie e opere d’arte.

In questo ecosistema si inserisce il *Datavault*, un database RDF (basato su Blazegraph) che funge da enorme bacino di immagazzinamento delle informazioni. Il Datavault

“contiene, ad ora, 19 milioni di triple tutte molto simili”, dotate di uno specifico “schema RDF reale”. Proprio a causa di questa vastità e similarità strutturale, “andare a cercare il file specifico non è cosa semplice ed immediata”: attualmente manca infatti un modo per “interrogare i dati per contenuto o concetto, nonostante Blazegraph renda l’informazione potenzialmente ricercabile in modo flessibile”.

3.2.2 Profilo dell’utente finale: i ricercatori del centro

Profilo disciplinare e competenze tecniche

L’utenza di riferimento del centro DH.ARC è composta da docenti, ricercatori, assegnisti e studenti di discipline prettamente umanistiche, quali filologia, storia, storia dell’arte e archeologia[PT22]. Questi studiosi sono molto competenti nel dominio delle fonti culturali e nell’analisi dei testi critici, ma tipicamente sono privi di un background informatico formale.

La loro alfabetizzazione digitale nel campo dell’Information Retrieval si limita solitamente all’impiego di strumenti “pronti all’uso”: cataloghi OPAC, database relazionali con maschere pre-configurate o motori di ricerca generici. Non hanno familiarità con triplestore RDF, SPARQL, o Knowledge Graphs.

Caratterizzazione dei bisogni informativi

In questo ecosistema, gli utenti non ricercano identificativi esatti o risposte chiuse; piuttosto, adottano comportamenti di ricerca esplorativa, procedendo per esplorazione libera al fine di collegare concetti e scoprire associazioni non immediate tra i documenti.

Interrogando il Datavault, gli umanisti del centro necessitano di porre interrogazioni cross-dominio articolate, formulando mentalmente domande come:

“Quali immagini di codici medievali del XIV secolo sono state digitalizzate con risoluzione superiore a 300 dpi nel corso del progetto X?”

o ancora:

“Mostrami tutti i manoscritti attribuiti a un determinato autore e digitalizzati nell’ultimo anno”.

3.3 Architettura Funzionale (Ad alto livello)

3.3.1 Il flusso dell’interazione

Dal punto di vista dell’utente, il sistema funziona secondo un flusso semplice:

1. **Input:** l’utente apre l’interfaccia del Datavault Browser, trova una barra di ricerca testuale (LLM Search Bar), e digita la sua domanda in italiano in forma libera. Non ci sono campi da compilare, sintassi da rispettare, o vocabolari controllati da imparare.

Esempio: “Mostrami tutte le immagini scansionate con una fotocamera Nikon nel 2022”.

2. **Elaborazione:** il testo naturale viene inviato a un componente intelligente (il modello LLM fine-tuned) che, guidato dalla conoscenza dell’ontologia del Datavault, produce una query SPARQL corrispondente.
3. **Output:** la query SPARQL viene eseguita sul Datavault (Blazegraph) e i risultati vengono restituiti e visualizzati nell’interfaccia come un albero di risorse navigabile. L’utente non vede mai la query SPARQL a meno che non la voglia; vede solo i risultati pertinenti alla sua domanda.

3.3.2 Diagramma funzionale del sistema

3.3.3 I componenti del sistema e le loro responsabilità

Il sistema si articola in tre componenti principali:

1. **Frontend (Datavault Browser):** interfaccia utente dove l’utente digita le domande e visualizza i risultati. Responsabile di: raccogliere input testuale, inviarlo al backend, visualizzare i risultati in formato navigabile.
2. **Backend (LLM Query Builder + RAG Engine):** il “cervello” del sistema. Responsabile di: ricevere la domanda naturale, recuperare lo schema RDF e predicati rilevanti dal Datavault, generare una query SPARQL utilizzando il modello LLM fine-tuned, gestire errori e allucinazioni.
3. **Data Layer (Blazegraph + Schema RDF):** il triplestore vero e proprio. Responsabile di: immagazzinare e gestire i 19 milioni di triple RDF, eseguire la query SPARQL generata dal backend, restituire i risultati al componente precedente.

3.4 Casi d’uso principali

Descriviamo tre scenari d’uso concreti che rappresentano i bisogni tipici della ricerca presso DH.ARC:

3.4.1 Caso d’uso 1: ricerca per caratteristiche di acquisizione

Attore: un ricercatore che vuole individuare immagini acquisite con specifiche caratteristiche tecniche.

Obiettivo: trovare tutte le immagini di una collezione digitalizzate con uno specifico dispositivo fotografico e in un dato intervallo temporale.

Input: “Mostrami tutte le immagini scansionate con una Nikon D850 tra il 2021 e il 2023”.

Traducibilità manuale in SPARQL: sarebbe complesso e richiederebbe conoscenza di `exif:make`, `exif:dateTime`, e gestione di range temporali.

Rilevanza: Questo caso d’uso dimostra come il sistema gestisce filtri tecnici (dispositivo fotografico) combinati con vincoli temporali, delegando all’LLM la complessità di tradurre una descrizione naturale in una query strutturata.

3.4.2 Caso d’uso 2: ricerca per progetto e tipologia di risorsa

Attore: un responsabile di un progetto di digitalizzazione che vuole fare un inventario delle risorse prodotte.

Obiettivo: elencare tutte le risorse di un progetto specifico con una determinata tipologia.

Input: “Elenca tutte le risorse del progetto NextScan caricate nel sistema”.

Traducibilità manuale in SPARQL: richiede conoscenza di predicati descrittivi (`dc:isPartOf`, `dc:title`) e della struttura dei progetti nel Datavault.

Rilevanza: Questo caso d’uso dimostra la capacità del sistema di filtrare per metadati descrittivi (Dublin Core) anziché tecnici (EXIF), rispondendo a un bisogno amministrativo frequente nel dominio DH.ARC.

3.4.3 Caso d’uso 3: ricerca multi-criterio con filtro temporale

Attore: un ricercatore che vuole analizzare la produzione digitale in un periodo specifico.

Obiettivo: trovare tutte le risorse acquisite in un intervallo temporale con ulteriori vincoli di qualità.

Input: “Mostrami tutte le risorse acquisite tra il 2022 e il 2023 con una risoluzione superiore a 4000 pixel”.

Traducibilità manuale in SPARQL: è il caso più complesso—dimostra la capacità del sistema di gestire query multi-criterio (filtri combinati su data E caratteristica tecnica), estremamente difficili da scrivere manualmente in SPARQL per un non esperto.

Rilevanza: Questo è il caso d’uso più forte per la nostra proposta: evidenzia il valore aggiunto del sistema rispetto ai filtri tradizionali e alla scrittura manuale di SPARQL.

3.5 Conclusioni del capitolo

In questo capitolo abbiamo presentato il prototipo di LLM Query Builder nel suo contesto operativo e funzionale. Abbiamo visto che:

1. Il dominio di applicazione (DH.ARC) presenta utenti con bisogni esplorativi complessi e assenza di competenze tecniche;
2. L’architettura funzionale del sistema è semplice dal punto di vista dell’utente: “domanda naturale → risultati”, nascondendo tutta la complessità nel backend;

3. I tre casi d'uso concreti dimostrano come il sistema riduce significativamente la barriera di accesso al Datavault.

Avendo chiarito il *cosa* e il *perché*, il capitolo 4 approfondisce le scelte architetturali e implementative che rendono possibile il funzionamento del sistema: la struttura del backend LLM, il processo di fine-tuning del modello, la gestione dell'ontologia RDF, e la progettazione del dataset di training.

Capitolo 4

Architettura e dettagli implementativi

4.1 Introduzione al capitolo

Questo capitolo illustra nel dettaglio l'architettura tecnica e le scelte implementative che costituiscono il motore dell'LLM Query Builder. Mentre il capitolo precedente ha descritto il sistema dal punto di vista dell'utente finale, qui verranno analizzati i componenti software, lo stack tecnologico e le metodologie di addestramento utilizzate per trasformare il linguaggio naturale in query SPARQL eseguibili.

La trattazione seguirà il flusso logico della costruzione del sistema: partendo dall'infrastruttura di base (paragrafo 4.2), passeremo alla complessa pipeline di fine-tuning del modello (paragrafo 4.3), per poi analizzare l'architettura del backend (paragrafo 4.4) e le specifiche del dataset di addestramento (paragrafo 4.5).

4.2 Stack tecnologico: Fedora e Blazegraph

L'infrastruttura dati su cui si appoggia l'LLM Query Builder è costituita dall'accoppiata Fedora Commons e Blazegraph. Blazegraph, in particolare, opera come triplestore ad alte prestazioni per il Datavault, fornendo l'endpoint SPARQL nativo interrogato dal nostro sistema. La sfida architettonica principale è consistita nel creare un layer intermedio capace di dialogare fluidamente con la rigidità formale richiesta da Blazegraph, astraendo completamente questa complessità dal frontend esposto al ricercatore.

4.3 Pipeline di fine-tuning dell'LLM

Per ottenere traduzioni accurate dal linguaggio naturale a SPARQL, l'utilizzo di un LLM generico si è rivelato insufficiente, rendendo necessario un processo di domain adaptation tramite fine-tuning.

4.3.1 Dataset generativo e schema SPARQL

Il cuore dell'addestramento è basato su un dataset generativo, prodotto tramite lo script `genera_dataset.py`. La versione finale (*v4*) comprende 16 categorie distinte di query, progettate per coprire l'intero spettro delle interrogazioni tipiche del dominio umanistico. Durante la fase di validazione, l'analisi delle chiamate di rete fallite aveva evidenziato un tasso di errore significativo dovuto a una scorretta generazione della sintassi SPARQL. Nello specifico, il modello tendeva a fallire a causa di un'errata gestione dei prefissi dei metadati. Per risolvere la problematica in modo strutturale, si è proceduto a raffinare gli esempi all'interno del dataset di addestramento, correggendo esplicitamente il modo in cui il modello mappa e dichiara i prefissi ontologici, e gestendo puntualmente i *type-mismatches* tra proprietà testuali e numeriche.

4.3.2 Configurazione dell'addestramento con Unsloth

Per ottimizzare i tempi di addestramento e ridurre l'impronta in memoria VRAM, la pipeline è stata implementata avvalendosi della libreria Unsloth. La configurazione ha previsto l'uso di precisione mista (`fp16` e `bf16`) e tecniche di *gradient checkpointing*. Inoltre, l'impiego del *data packing* ha permesso di concatenare sequenze più brevi per massimizzare l'efficienza computazionale durante l'aggiornamento dei pesi.

4.4 Architettura del backend e integrazione frontend

4.4.1 Inference engine (FastAPI)

L'orchestrazione delle richieste è demandata a un backend sviluppato in Python tramite il framework FastAPI. L'inference engine è stato calibrato per garantire la consistenza del *system prompt* a ogni invocazione, vincolando il modello a mantenere il ruolo di traduttore tecnico. Particolare attenzione è stata posta alla corretta generazione del token EOS (*End Of Sentence*), fondamentale per troncare l'output del modello ed evitare allucinazioni aggiuntive in coda alla query SPARQL.

Lo sviluppo iterativo di questi componenti in Python e del frontend in React è stato ampiamente facilitato dall'impiego di ambienti di sviluppo avanzati basati su agenti, come Antigravity IDE, che hanno permesso di ottimizzare la stesura del codice e testare rapidamente le configurazioni dei prompt di sistema direttamente all'interno della codebase.

4.4.2 Integrazione con Blazegraph tramite proxy Vite

Sul versante client, l'integrazione è gestita tramite un proxy configurato in Vite. Questa architettura previene problemi legati alle policy CORS e instrada le query SPARQL generate direttamente dall'interfaccia React all'endpoint di Blazegraph, garantendo un flusso dati bidirezionale sicuro e performante tra l'utente e il Datavault.

4.5 Formato dati: `dataset__datavault__v4.jsonl`

4.5.1 Le 8 regole di generazione SPARQL

Il formato dati alla base del fine-tuning, consolidato nel file `dataset_datavault_v4.jsonl`, struttura la conoscenza del dominio in formato JSONL. Per istruire l'LLM, sono state codificate 8 specifiche regole di generazione SPARQL. Queste regole impongono al modello restrizioni rigorose su quali relazioni attraversare, come filtrare le date e come gestire la paginazione e i limiti dei risultati.

4.5.2 Gestione degli errori: `OUT_OF_DOMAIN`

Un requisito fondamentale per l'affidabilità in un contesto accademico è la capacità del sistema di ammettere i propri limiti. Nel dataset è stata implementata un'architettura di *error handling* JSON-based: quando l'utente pone una domanda irrilevante rispetto allo schema del Datavault, il modello è addestrato a non tentare di forzare una query SPARQL, ma a restituire un esplicito stato `OUT_OF_DOMAIN`. Questo fallback previene l'esecuzione di query malformate e migliora la trasparenza dell'interfaccia utente.

4.6 Conclusioni del capitolo

Le scelte architetturali descritte in questo capitolo — dal raffinamento del dataset per i prefissi, fino all'implementazione del backend FastAPI — dimostrano come la transizione da un proof-of-concept teorico a uno strumento funzionante richieda un'attenta ingegnerizzazione del dato e del software. Nel capitolo 5 esamineremo come questa architettura si comporta concretamente in fase di test quantitativo e qualitativo.

Capitolo 5

Valutazione

5.1 Introduzione all'approccio valutativo

5.2 Valutazione dell'efficienza (Quantitativa)

5.2.1 Ambienti di test: GPU vs CPU

5.2.2 Impronta in memoria e throughput

5.3 Valutazione dell'efficacia (Qualitativa)

5.3.1 Precision e Recall su Blazegraph

5.3.2 Analisi del tasso di allucinazione

5.3.3 Robustezza sui casi limite (Edge Cases)

5.4 Conclusioni del capitolo

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Risultati raggiunti

6.2 Motivazioni di orgoglio e modestia

6.2.1 Punti di forza e orgoglio

6.2.2 Limiti del sistema e punti di modestia

6.3 Sviluppi futuri

6.3.1 Bug fix e ottimizzazioni a breve termine

6.3.2 Estensioni fondamentali e feature mancanti

6.3.3 Visioni di lungo termine (Desideri e sogni)

Riferimenti bibliografici

- [ALM26] ALMA MATER STUDIORUM - Università di Bologna. /*DH.ARC - Digital Humanities Advanced Research Centre*. <https://centri.unibo.it/dharc/en>, 2026.
- [Bat89] Marcia J. Bates. The design of browsing and berrypicking techniques for the online search interface. *Online Review*, 13(5):407–424, 1989.
- [Mar25] Mohammed Maree. Quantifying relational exploration in cultural heritage knowledge graphs with llms: A neuro-symbolic approach for enhanced knowledge discovery. *Data*, 10(4):52, 2025.
- [MR08] Peter Morville and Louis Rosenfeld. *Information Architecture for the World Wide Web*. O'Reilly Media, Sebastopol, CA, 3 edition, 2008.
- [PT22] Silvio Peroni and Francesca Tomasi. Approaching digital humanities at university: a cultural challenge. *Journal of Art Historiography*, 27, December 2022.
- [Ros07] Luca Rosati. *Architettura dell'informazione: Guida alla trovabilità, dagli oggetti quotidiani al Web*. Apogeo, 2007.
- [Tol70] J. R. R. Tolkien. *Il Signore degli Anelli*. Bompiani, 1970.
- [Vaš24] Linas Vaštakas. Cultural heritage search with large language models: Enhancing the discoverability of cultural heritage artifacts through large language model-based search systems. Master's thesis, Linnaeus University, Sweden, 2024.
- [VFQP25] Iva Vasic, Hans-Georg Fill, Ramona Quattrini, and Roberto Pierdicca. Knowledge graphs vs. large language models: Competitors or partners in supporting virtual museums. *ACM Journal on Computing and Cultural Heritage*, 18(4), dec 2025.

Ringraziamenti

Grazie a tutt*.